

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 - CONCEPT MAPPING

Course Code:	CSA1205	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	C01	Assessment:	ASSESSMENT TOOL 2- CONCEPT MAPPING
Weightage:	35%	Total Marks:	25
C01 Statement:	Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)		
Student Name:	P.Harsha vardhan	Reg.No.:	192525464
Department:		Date:	14-7-26

Instruction to Students

1. Answer two questions as per the Instruction.
2. Each question carries 12.5 mark.
3. Only the appropriate Tools can be used
4. Time allotted for the exam is 60 min

ANNEXURE A

1. Construct a detailed concept map showing the relationship between CPU (ALU, Control Unit, Registers), memory (cache, RAM), and I/O devices. Extend the map to include single-bus, multi-bus, and hierarchical bus structures, and clearly illustrate how data and control signals flow among these components during execution. Indicate possible points of delay or bottlenecks and how different bus structures help in improving performance.
 2. Develop a comprehensive concept map linking the instruction execution cycle (fetch, decode, execute) with CPU performance parameters such as CPI, clock cycles, and execution time. Show how each stage contributes to overall execution time, and include connections to factors like memory access, instruction complexity, and pipeline stalls. Also, represent how benchmarking is used to evaluate performance.
 3. Create a concept map comparing 8085 and 8086 architectures, including their instruction sets, register organization, addressing modes, and memory segmentation (in 8086). Clearly show how these architectural features influence program execution, multitasking capability, and efficiency in real-time applications.
 4. Draw a detailed concept map that connects various addressing modes (immediate, direct, register, indirect, indexed) with instruction types (data transfer, arithmetic, control instructions). Illustrate how each addressing mode affects memory access time, instruction size, and execution speed, and show real-world scenarios where specific addressing modes are preferred.
 5. Prepare an elaborate concept map integrating number systems (binary and hexadecimal) with instruction representation, memory storage, and debugging processes. Show how data is converted between formats, how instructions are stored and interpreted by the CPU, and why hexadecimal representation is commonly used in low-level programming and system design.
-

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

Criteria	Concept Identification (25%)	Linkages (25%)	Organization (20%)	Integration of Literature (20%)	Clarity (10%)
Q1					
Q2					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

Question 1: CPU, Memory, I/O Devices and Bus Structures, Construct a detailed concept map showing the relationship between CPU (ALU, Control Unit, Registers), memory (cache, RAM), and I/O devices. Extend the map to include single-bus, multi-bus, and hierarchical bus structures, and clearly illustrate how data and control signals flow among these components during execution. Indicate possible points of delay or bottlenecks and how different bus structures help in improving performance

Answer

A computer system is made up of three important components: the **Central Processing Unit (CPU)**, **Memory**, and **Input/Output (I/O) devices**. The CPU acts as the brain of the computer and is responsible for executing all instructions. It consists of the **Arithmetic Logic Unit (ALU)** for performing arithmetic and logical operations, the **Control Unit (CU)** for controlling instruction execution, and **Registers** for temporarily storing data during processing.

Memory is divided into **Cache Memory** and **RAM**. Cache memory is small but very fast and stores frequently used data, while RAM stores programs and data that are currently in use. Input devices such as the keyboard and mouse send data to the CPU, whereas output devices like the monitor and printer display the processed results. Communication between the CPU, memory, and I/O devices takes place through the **system bus**, which includes the **data bus**, **address bus**, and **control bus**. A **single-bus structure** is simple but can become a bottleneck because all components share the same bus. A **multi-bus structure** allows multiple data transfers simultaneously, reducing delays and improving system performance. A **hierarchical bus structure** organizes buses into different levels, enabling faster communication and efficient expansion of computer systems.

Sample C Program

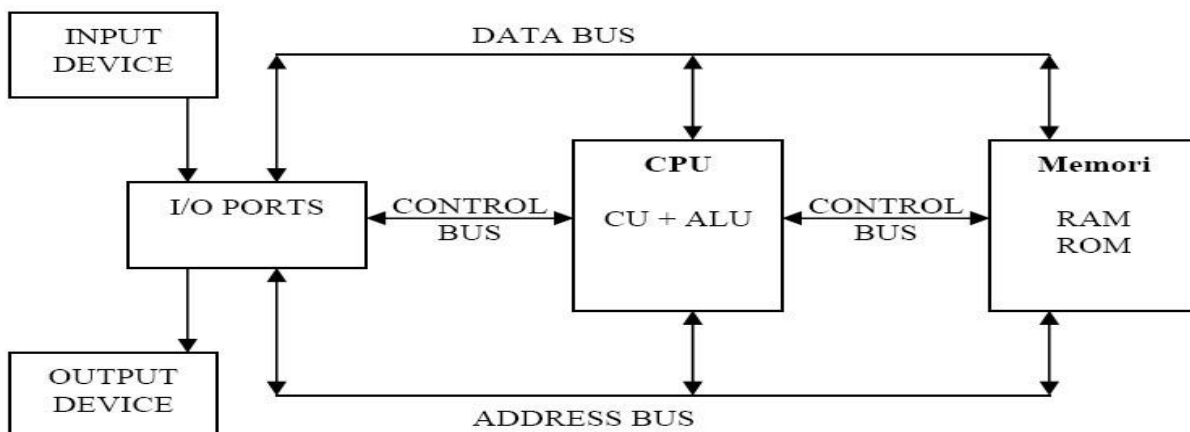
```
#include<stdio.h>
```

```
int main()
{
    int a=25,b=15;
    printf("Addition = %d",a+b);
    return 0;
}
```

Out put:

Addition = 40

Flowchart (Image):



Question 2: Instruction Execution Cycle and CPU Performance

Answer

The **instruction execution cycle** is the process followed by the CPU to execute every instruction in a program. It consists of three major stages: **Fetch, Decode, and Execute**.

During the **Fetch** stage, the CPU retrieves an instruction from memory using the Program Counter (PC). In the **Decode** stage, the Control Unit interprets the instruction and determines the required operation. Finally, in the **Execute** stage, the ALU performs the operation, and the result is stored either in a register or memory.

The performance of the CPU depends on factors such as **Clock Cycles, Cycles Per Instruction (CPI), Execution Time, Memory Access Time, and Pipeline Stalls**. A processor with a lower CPI and fewer pipeline stalls executes instructions more efficiently. **Benchmarking** is used to compare processor performance by measuring the execution time of standard programs.

The CPU execution time is calculated using the formula:

$$\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

Reducing memory delays and improving pipelining significantly enhance overall processor performance.

Sample C Program

```
#include<stdio.h>
```

```
int main()
{
    int x=20,y=10;

    printf("Sum = %d\n",x+y);
    printf("Difference = %d",x-y);

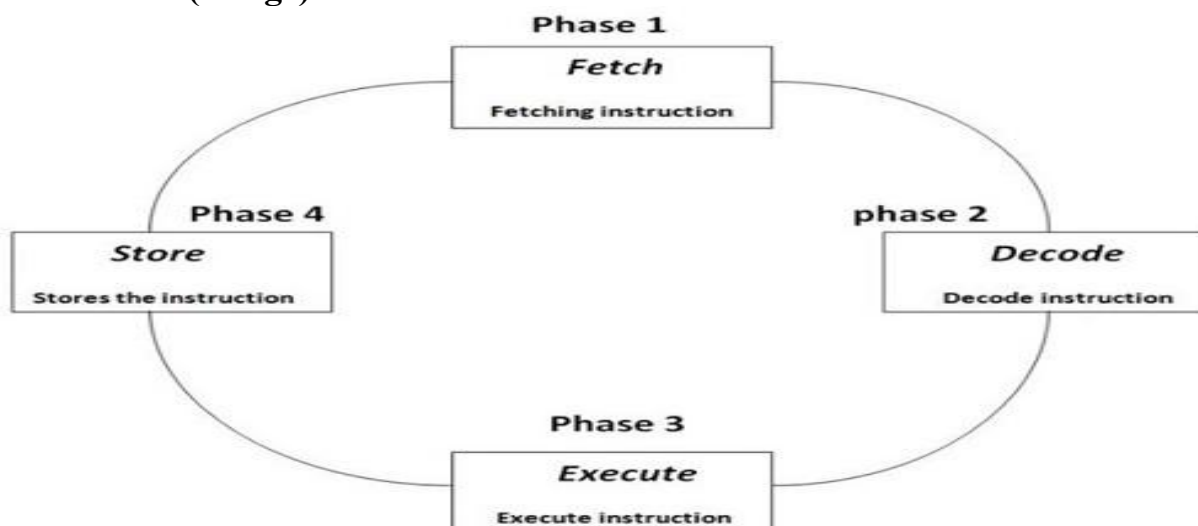
    return 0;
}
```

Output

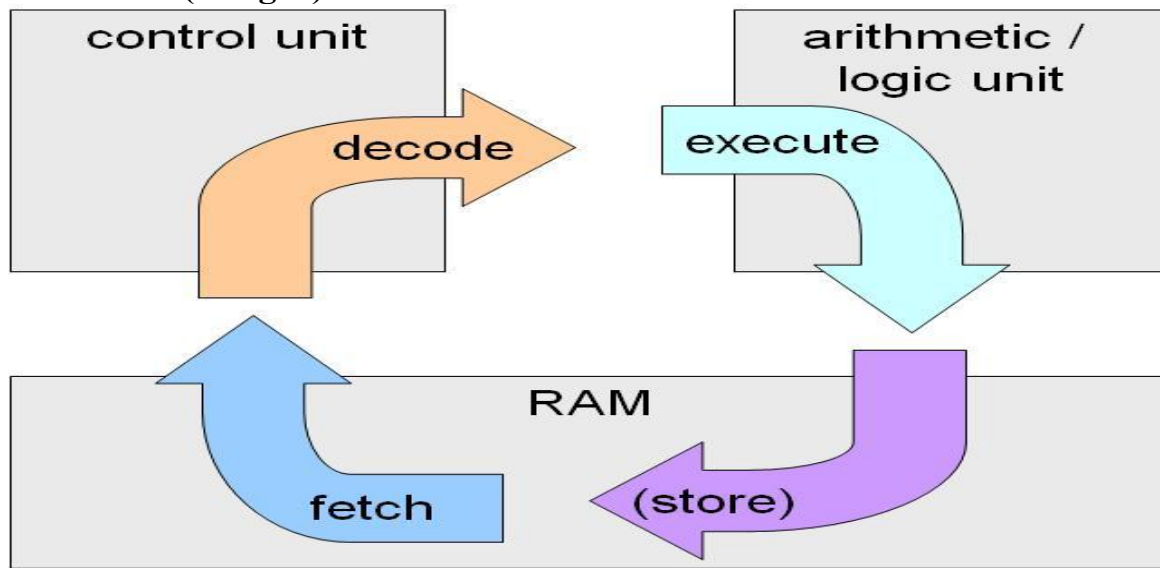
Sum = 30

Difference = 10

Flowchart (Image):



Flowchart (Image2):



Conclusion:

The Instruction Execution Cycle (Fetch–Decode–Execute) forms the foundation of CPU operation, with each stage directly influencing overall system performance. The fetch stage depends on efficient memory access, the decode stage is affected by instruction complexity, and the execute stage determines how quickly operations are completed. These factors collectively impact Clock Cycles Per Instruction (CPI), the total clock cycles, and ultimately the CPU execution time. Pipeline stalls, cache misses, and complex instructions can increase CPI and reduce performance, while optimized pipelines and faster memory improve efficiency. Benchmarking plays a crucial role by measuring and comparing CPU performance under standardized workloads, helping evaluate how effectively a processor executes instructions in real-world scenarios.